# CSA4704-DEEP LEARNING

**NAME:HARITHI S** **REG NO:192421151**

Experiment 13: Sequential Dataset Preparation and RNN Prediction

Problem Statement: A state electricity board must forecast next-hour power demand to schedule generation and avoid shortages. Hourly electricity consumption data for one year (or a daily temperature series as a substitute). The objective is to convert a time series into supervised sequences and predict the next value with a SimpleRNN. Use TensorFlow, Keras, NumPy, Pandas, Matplotlib.

Tasks:

1. Load the series with Pandas; handle missing values and scale it.
2. Create sliding windows (for example, 24 past hours → next hour).
3. Split chronologically into train and test sets.
4. Build and train a SimpleRNN model in Keras.
5. Plot actual against predicted demand and report MAE and RMSE.

**FORMULA:**

**1. Min-Max Normalisation**

$X_scaled = (X - X_min) / (X_max - X_min)$

**2. Sliding Window Sequence**

$X□ = [x□_{-23}, x□_{-22}, ..., x□]$

$Y□ = x□_{+1}$

**3. SimpleRNN Hidden State**

$h□ = \tanh(W_{x}h\ X□ + W□h\ h□_{-1} + b□)$

**4. Predicted Output**

$ŷ□ = W□_{\gamma}\ h□ + b_{\gamma}$

**5. Mean Absolute Error (MAE)**

$MAE = (1/N) \Sigma |y_i - \hat{y}_i|$

**6. Mean Squared Error (MSE)**

$MSE = (1/N) \Sigma (y_i - \hat{y}_i)^2$

**7. Root Mean Squared Error (RMSE)**

$RMSE = \sqrt{[(1/N) \Sigma (y_i - \hat{y}_i)^2]}$

**8. Training and Testing Split**

Training Data = First 80% of the time series

Testing Data = Remaining 20% of the time series

**CODE:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

# 1. Generate sample hourly electricity demand data
np.random.seed(42)
hours = np.arange(24 * 365)
demand = (100 + 20 * np.sin(2 * np.pi * hours / 24)
        + np.random.normal(0, 5, len(hours)))

df = pd.DataFrame({"Demand": demand})
df["Demand"] = df["Demand"].interpolate().bfill()

# 2. Split chronologically before scaling
split_point = int(len(df) * 0.8)
train_data = df["Demand"].iloc[:split_point].values.reshape(-1, 1)
test_data = df["Demand"].iloc[split_point:].values.reshape(-1, 1)
```

```
scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train_data)
all_scaled = scaler.transform(df[["Demand"]])

# 3. Create sliding windows: 24 hours -> next hour
def create_sequences(data, window=24):
    X, y = [], []
    for i in range(window, len(data)):
        X.append(data[i-window:i])
        y.append(data[i])
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_scaled)

# Include the last 24 training hours as context for test predictions
X_test, y_test = [], []
for i in range(split_point, len(all_scaled)):
    X_test.append(all_scaled[i-24:i])
    y_test.append(all_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)
X_test, y_test = np.array(X_test), np.array(y_test)

# 4. Build and train SimpleRNN
model = Sequential([
    tf.keras.Input(shape=(24, 1)),
    SimpleRNN(32, activation="tanh"),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse")
model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=1)

# 5. Predict and reverse scaling
predictions = model.predict(X_test, verbose=0)
actual = scaler.inverse_transform(y_test)
predicted = scaler.inverse_transform(predictions)

# 6. Calculate MAE and RMSE
```

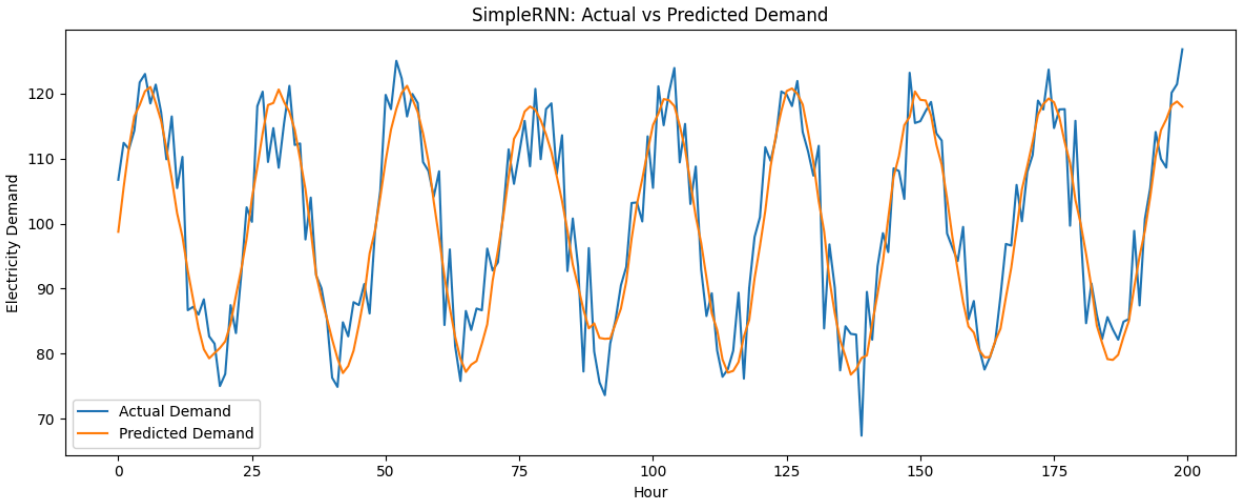
```
mae = mean_absolute_error(actual, predicted)
rmse = np.sqrt(mean_squared_error(actual, predicted))

print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))

# 7. Plot actual vs predicted demand
plt.figure(figsize=(12, 5))
plt.plot(actual[:200], label="Actual Demand")
plt.plot(predicted[:200], label="Predicted Demand")
plt.xlabel("Hour")
plt.ylabel("Electricity Demand")
plt.title("SimpleRNN: Actual vs Predicted Demand")
plt.legend()
plt.tight_layout()
plt.show()
```

**OUTPUT:**

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but MinMaxScaler was fitted without feature names
  warnings.warn(
Epoch 1/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - loss: 0.0269
Epoch 2/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0065
Epoch 3/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0063
Epoch 4/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0062
Epoch 5/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.0062
Epoch 6/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0061
Epoch 7/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0061
Epoch 8/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0061
Epoch 9/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0061
Epoch 10/10
219/219 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.0060
MAE: 4.43
RMSE: 5.59
```


SimpleRNN: Actual vs Predicted Demand
Electricity Demand
Hour
Actual Demand
Predicted Demand
120
110
100
90
80
70
0
25
50
75
100
125
150
175
200